

CSS Flexbox Order

Complete Guide: From Basics to Advanced

Contents

1	Introduction	2
1.1	What is the Order Property?	2
1.2	Why Use the Order Property?	2
1.3	Order vs. Position	2
1.4	Default Order Value	3
2	Understanding Order	4
2.1	How Order Works	4
2.2	Order Values	4
2.2.1	Valid Values	4
2.2.2	Comparison	4
2.3	The Ordering Algorithm	5
3	Basic Examples	6
3.1	Example 1: Reordering Navigation	6
3.2	Example 2: Mobile-First Reordering	6
3.3	Example 3: Featured Item First	7
3.4	Example 4: Carousel-Like Reordering	8
4	Practical Applications	9
4.1	Application 1: Responsive Layout	9
4.2	Application 2: Hero Section Reordering	10
4.3	Application 3: Dashboard Reordering	10
4.4	Application 4: E-Commerce Product Page	11
5	Advanced Techniques	13
5.1	CSS Variables with Order	13
5.2	Using nth-child with Order	13
5.3	Dynamic Order with Data Attributes	14
5.4	Combining Order with Media Queries	14
6	Responsive Patterns	15
6.1	Mobile-First Pattern	15
6.2	Content-First Approach	15
7	Accessibility Considerations	17
7.1	Important: Order and Accessibility	17
7.2	Best Practices for Accessibility	17

7.3	Example: Proper Order Usage	18
8	Troubleshooting	19
8.1	Issue 1: Order Not Working	19
8.2	Issue 2: Unexpected Sort Order	19
8.3	Issue 3: Accessibility Issues	20
9	Best Practices	21
9.1	When to Use Order	21
9.2	When NOT to Use Order	21
9.3	Code Organization	21
9.4	Performance Tips	22
10	Summary: Key Takeaways	23
10.1	Core Concepts	23
10.2	Key Rules	23
10.3	Quick Reference	24
10.4	Practical Ordering Strategy	24
10.5	Common Use Cases	24
10.6	Comparison: Order Properties	25

Chapter 1

Introduction

1.1 What is the Order Property?

The **order** property controls the visual order of flex items without changing the HTML structure. Items with lower order values appear first, while items with higher values appear last.

Key Point: The **order** property is purely visual. It affects display order only, not the DOM order or logical reading order for accessibility.

1.2 Why Use the Order Property?

- Reorder items without changing HTML
- Create responsive layouts easily
- Maintain semantic HTML structure
- Highlight important content
- Adapt layouts for different screen sizes
- Simplify CSS complexity
- No extra markup needed

1.3 Order vs. Position

Understanding the difference is crucial:

Order (Flexbox)	Position (Absolute)
Purely visual ordering	Moves elements spatially
Maintains flow layout	Takes out of flow
Reorders within layout	Overlapping possible
Simple and clean	More complex
Semantic HTML preserved	DOM order less relevant

1.4 Default Order Value

All flex items have a default `order` value of 0. Items are rendered in source order by default.

Chapter 2

Understanding Order

2.1 How Order Works

```
/* All items have default order: 0 */
.flex-item {
  order: 0; /* Default */
}

/* Items render in source order when order is equal */
/* So Item 1, Item 2, Item 3 by default */

/* Change order to reorder */
.item-1 { order: 3; } /* Appears last */
.item-2 { order: 1; } /* Appears first */
.item-3 { order: 2; } /* Appears middle */

/* Visual order: Item 2, Item 3, Item 1 */
```

2.2 Order Values

2.2.1 Valid Values

- Any integer (positive or negative)
- Default: 0
- Common range: -1000 to 1000
- Most common: Small integers like -1, 0, 1, 2

2.2.2 Comparison

```
/* All valid order values */
.item-1 { order: -10; } /* Very high priority (first) */
.item-2 { order: -1; } /* High priority */
.item-3 { order: 0; } /* Default */
.item-4 { order: 1; } /* Lower priority */
.item-5 { order: 100; } /* Very low priority (last) */
```

```
/* Visual order: -10, -1, 0, 1, 100 */
```

2.3 The Ordering Algorithm

1. Items sorted by **order** value (ascending)
2. Items with same order value follow source order
3. Flex container respects the sorted order
4. Layout proceeds normally

Chapter 3

Basic Examples

3.1 Example 1: Reordering Navigation

```
<nav class="navbar">
  <a href="#" class="home">Home</a>
  <a href="#" class="about">About</a>
  <a href="#" class="services">Services</a>
  <a href="#" class="contact">Contact</a>
</nav>

/* Default order in HTML */
/* Visual: Home About Services Contact */

.navbar {
  display: flex;
  gap: 20px;
  background: #2c3e50;
  padding: 15px;
}

.navbar a {
  color: white;
  text-decoration: none;
}

.contact { order: -1; } /* Contact first */
.home { order: 0; } /* Home second (default) */
.about { order: 1; } /* About third */
.services { order: 2; } /* Services last */

/* Visual order: Contact Home About Services */
```

3.2 Example 2: Mobile-First Reordering

```
<div class="layout">
  <header class="header">Header</header>
  <nav class="nav">Navigation</nav>
```



```
<main class="main">Main Content</main>
<aside class="sidebar">Sidebar</aside>
<footer class="footer">Footer</footer>
</div>

/* Desktop order */
.layout {
  display: flex;
  flex-direction: column;
  gap: 20px;
}

.header { order: 1; }
.nav { order: 2; }
.main { order: 3; }
.sidebar { order: 4; }
.footer { order: 5; }

/* Mobile order - reorder for small screens */
@media (max-width: 768px) {
  .header { order: 1; } /* Stays first */
  .nav { order: 2; } /* Stays second */
  .sidebar { order: 3; } /* Move sidebar up */
  .main { order: 4; } /* Main after sidebar */
  .footer { order: 5; } /* Stays last */
}
```

3.3 Example 3: Featured Item First

```
<div class="products">
  <div class="product">Product 1</div>
  <div class="product featured">Featured</div>
  <div class="product">Product 3</div>
  <div class="product">Product 4</div>
</div>

.products {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}

.product {
  flex: 1 1 200px;
  background: white;
  padding: 20px;
  border-radius: 8px;
  order: 0; /* Default */
}

.product.featured {
  order: -1; /* Featured appears first */
}
```

```
    flex: 1 1 100%; /* Full width */
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
}

/* Featured product appears first regardless of HTML position */
```

3.4 Example 4: Carousel-Like Reordering

```
<div class="carousel">
  <div class="slide" data-order="1">Slide 1</div>
  <div class="slide" data-order="2">Slide 2</div>
  <div class="slide" data-order="3">Slide 3</div>
  <div class="slide" data-order="4">Slide 4</div>
</div>

.carousel {
  display: flex;
  gap: 20px;
  overflow: hidden;
}

.slide {
  flex: 0 0 100%;
  background: #3498db;
  color: white;
  padding: 40px;
  text-align: center;
}

/* Use JavaScript to rotate order values */
/* .slide:nth-child(1) { order: 2; } */
/* .slide:nth-child(2) { order: 3; } */
/* .slide:nth-child(3) { order: 4; } */
/* .slide:nth-child(4) { order: 1; } */
```

Chapter 4

Practical Applications

4.1 Application 1: Responsive Layout

```
<div class="page">
  <aside class="sidebar">Sidebar</aside>
  <main class="content">Main Content</main>
  <article class="featured">Featured Post</article>
</div>

/* Desktop: Sidebar, Main, Featured */
.page {
  display: flex;
  gap: 20px;
}

.sidebar {
  order: 1;
  width: 250px;
}

.content {
  order: 2;
  flex: 1;
}

.featured {
  order: 3;
  width: 250px;
}

/* Mobile: Featured, Main, Sidebar */
@media (max-width: 768px) {
  .page {
    flex-direction: column;
  }

  .featured { order: 1; } /* Featured on top */
  .content { order: 2; }
  .sidebar { order: 3; width: 100%; }
```

```
}
```

4.2 Application 2: Hero Section Reordering

```
<section class="hero">
  <div class="text">
    <h1>Welcome</h1>
    <p>Description here</p>
  </div>
  <div class="image">
    
  </div>
</section>

/* Desktop: Text on left, image on right */
.hero {
  display: flex;
  align-items: center;
  gap: 40px;
}

.text { order: 1; flex: 1; }
.image { order: 2; flex: 1; }

/* Mobile: Image on top, text below */
@media (max-width: 768px) {
  .hero {
    flex-direction: column;
  }

  .image { order: 1; } /* Image first */
  .text { order: 2; }
}
```

4.3 Application 3: Dashboard Reordering

```
<div class="dashboard">
  <div class="card stats">Stats</div>
  <div class="card chart">Chart</div>
  <div class="card table">Table</div>
  <div class="card summary">Summary</div>
</div>

/* Desktop: Stats, Chart, Table, Summary */
.dashboard {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}
```

```
.card {
  flex: 1 1 250px;
  background: white;
  padding: 20px;
}

.card.stats { order: 1; }
.card.chart { order: 2; }
.card.table { order: 3; }
.card.summary { order: 4; }

/* Mobile: Summary first (important), then others */
@media (max-width: 768px) {
  .card.summary { order: 1; } /* Most important first */
  .card.stats { order: 2; }
  .card.chart { order: 3; }
  .card.table { order: 4; }
}
```

4.4 Application 4: E-Commerce Product Page

```
<div class="product-page">
  <div class="images">Product Images</div>
  <div class="details">Product Details</div>
  <div class="reviews">Reviews</div>
  <div class="related">Related Products</div>
</div>

/* Desktop: Images left, Details right, Reviews below */
.product-page {
  display: flex;
  flex-wrap: wrap;
  gap: 30px;
}

.images {
  order: 1;
  flex: 0 0 45%;
}

.details {
  order: 2;
  flex: 0 0 45%;
}

.reviews {
  order: 3;
  flex: 1;
}

.related {
  order: 4;
}
```

```
    flex: 1;
}

/* Mobile: Different priority */
@media (max-width: 768px) {
    .images { order: 1; flex: 1; }      /* Images first */
    .details { order: 2; flex: 1; }     /* Details */
    .related { order: 3; flex: 1; }     /* Related before reviews */
    .reviews { order: 4; flex: 1; }
}
```

Chapter 5

Advanced Techniques

5.1 CSS Variables with Order

```
:root {
  --order-header: 1;
  --order-main: 2;
  --order-sidebar: 3;
  --order-footer: 4;
}

.header { order: var(--order-header); }
.main { order: var(--order-main); }
.sidebar { order: var(--order-sidebar); }
.footer { order: var(--order-footer); }

/* Easy to adjust all orders */
@media (max-width: 768px) {
  :root {
    --order-sidebar: 2; /* Sidebar comes earlier */
    --order-main: 3;
  }
}
```

5.2 Using nth-child with Order

```
.grid {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}

.item {
  flex: 1 1 200px;
}

/* Reverse visual order */
.item:nth-child(1) { order: 4; }
```

```
.item:nth-child(2) { order: 3; }
.item:nth-child(3) { order: 2; }
.item:nth-child(4) { order: 1; }

/* Items appear in reverse */
```

5.3 Dynamic Order with Data Attributes

```
<div class="flex-container">
  <div class="item" data-priority="3">Item 1</div>
  <div class="item" data-priority="1">Item 2</div>
  <div class="item" data-priority="2">Item 3</div>
</div>

.item[data-priority="1"] { order: 1; }
.item[data-priority="2"] { order: 2; }
.item[data-priority="3"] { order: 3; }

/* Order by data attribute */
```

5.4 Combining Order with Media Queries

```
.layout {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}

.section { flex: 1 1 100%; }

/* Small screens: Priority order */
@media (max-width: 480px) {
  .featured { order: 1; }
  .primary { order: 2; }
  .secondary { order: 3; }
}

/* Medium screens: Different order */
@media (min-width: 481px) and (max-width: 768px) {
  .primary { order: 1; }
  .featured { order: 2; }
  .secondary { order: 3; }
}

/* Large screens: Default order */
@media (min-width: 769px) {
  .primary { order: 1; }
  .secondary { order: 2; }
  .featured { order: 3; }
}
```


Chapter 6

Responsive Patterns

6.1 Mobile-First Pattern

```
/* Mobile: Simplified order */
.container {
  display: flex;
  flex-direction: column;
}

.item { order: 0; }
.featured { order: 0; } /* Same level */

/* Tablet: Some reordering */
@media (min-width: 768px) {
  .container {
    flex-direction: row;
    flex-wrap: wrap;
  }

  .featured { order: -1; } /* Featured first */
  .item { order: 0; }
}

/* Desktop: Full reordering */
@media (min-width: 1200px) {
  .featured { order: 2; } /* Featured in middle */
  .primary { order: 1; }
  .secondary { order: 3; }
}
```

6.2 Content-First Approach

```
/* Prioritize content visibility on mobile */
.page {
  display: flex;
  flex-direction: column;
}
```

```
.main-content { order: 1; } /* Content first on mobile */
.sidebar { order: 2; }
.header { order: 3; }

/* Desktop: Traditional order */
@media (min-width: 1024px) {
  .page {
    flex-direction: row;
  }

  .header { order: 1; }
  .sidebar { order: 2; }
  .main-content { order: 3; }
}
```

Chapter 7

Accessibility Considerations

7.1 Important: Order and Accessibility

```
/* IMPORTANT: Order is visual only! */
/* HTML source order affects:
   - Screen readers
   - Keyboard navigation
   - Search engine crawlers
*/

.nav {
    display: flex;
}

.nav-item { order: 0; }

.nav-item.contact {
    order: -1; /* Appears first visually */
}

/* BUT: Keyboard users still navigate in HTML order */
/* Screen readers still follow HTML order */
```

7.2 Best Practices for Accessibility

- Keep HTML source order logical
- Don't use order to hide important content
- Test with keyboard navigation
- Test with screen readers
- Ensure visual order makes sense
- Consider cognitive load
- Document order changes

7.3 Example: Proper Order Usage

```
<!-- HTML: Logical source order -->
<nav class="navigation">
  <a href="/" class="home">Home</a>
  <a href="/about" class="about">About</a>
  <a href="/contact" class="contact">Contact</a>
</nav>

<!-- CSS: Visual reordering is fine -->
.navigation {
  display: flex;
}

.home { order: 1; }
.about { order: 2; }
.contact { order: 3; }

<!-- This is accessible because HTML order is logical -->
```

Chapter 8

Troubleshooting

8.1 Issue 1: Order Not Working

Problem: order property has no effect.

Solution: Ensure display: flex is set on parent.

```
/* WRONG: No flex display */
.container {
  order: 1; /* Has no effect */
}

/* CORRECT: Set display: flex on parent */
.container {
  display: flex;
}

.item {
  order: 1; /* Works now */
}
```

8.2 Issue 2: Unexpected Sort Order

Problem: Items appear in unexpected visual order.

Solution: Check order values and source order for ties.

```
/* Items with same order follow source order */
.item-1 { order: 1; }
.item-2 { order: 1; } /* Same as item-1 */
.item-3 { order: 1; } /* Same as item-1 */

/* Result: item-1, item-2, item-3 (HTML order) */

/* Explicit different order values */
.item-1 { order: 1; }
.item-2 { order: 2; }
.item-3 { order: 3; }

/* Result: item-1, item-2, item-3 (explicit order) */
```

8.3 Issue 3: Accessibility Issues

Problem: Visual order differs from logical order.

Solution: Keep HTML source order logical.

```
<!-- Good: Logical HTML order -->
<div class="flex">
  <h1>Title</h1>
  <p>Description</p>
  <button>Action</button>
</div>

<!-- CSS: Visual reordering is fine -->
h1 { order: 2; }
p { order: 1; }
button { order: 3; }

<!-- Screen readers still get logical order -->
```

Chapter 9

Best Practices

9.1 When to Use Order

- Create responsive layouts
- Reorder content for different screen sizes
- Highlight important items
- Adapt visual hierarchy
- Simplify HTML structure
- Avoid duplicate content

9.2 When NOT to Use Order

- To fix illogical HTML structure
- To completely reverse content order
- When it conflicts with accessibility
- For positioning (use other properties)
- To hide content from users

9.3 Code Organization

```
/* GOOD: Clear order assignments */
.primary { order: 1; }
.secondary { order: 2; }
.tertiary { order: 3; }

/* BAD: Confusing order values */
.section-1 { order: 999; }
.section-2 { order: 1; }
.section-3 { order: 500; }
```

```
/* GOOD: Use CSS variables */
:root {
  --order-1: 1;
  --order-2: 2;
  --order-3: 3;
}

.primary { order: var(--order-1); }
```

9.4 Performance Tips

- Order is performant - use freely
- Browser handles efficiently
- No JavaScript needed for basic reordering
- Combine with media queries
- Test at different breakpoints

Chapter 10

Summary: Key Takeaways

10.1 Core Concepts

- `order` controls visual item order only
- Default value is 0
- Lower values appear first
- Accepts any integer
- Works on cross-axis items
- HTML order unaffected

10.2 Key Rules

1. Default `order` is 0
2. Lower order values appear first
3. Items with same order follow source order
4. Negative values place items before defaults
5. HTML structure unaffected
6. Accessibility maintained if used properly
7. Perfect for responsive design
8. Responsive with media queries
9. Very efficient and performant
10. Keep values simple and organized

10.3 Quick Reference

```
/* Reorder items */
.item-1 { order: 3; }
.item-2 { order: 1; }
.item-3 { order: 2; }

/* Featured first */
.featured { order: -1; }
.normal { order: 0; }

/* Responsive reordering */
.item { order: 1; }

@media (max-width: 768px) {
  .featured { order: 1; }
  .item { order: 2; }
}

/* Using negative values */
.important { order: -1; }
.less-important { order: 1; }
```

10.4 Practical Ordering Strategy

```
/* Priority-based ordering */
:root {
  --priority-critical: -10;
  --priority-high: -1;
  --priority-normal: 0;
  --priority-low: 1;
}

.critical { order: var(--priority-critical); }
.high { order: var(--priority-high); }
.normal { order: var(--priority-normal); }
.low { order: var(--priority-low); }
```

10.5 Common Use Cases

- Mobile-first responsive layouts
- Feature emphasis on different devices
- Navigation reordering
- Content priority for mobile
- Hero section repositioning
- Sidebar movement

- Dashboard reorganization
- Featured content highlighting

10.6 Comparison: Order Properties

Property	Purpose
order	Visual reordering
z-index	Layering/stacking
position	Spatial positioning
margin	Spacing/alignment